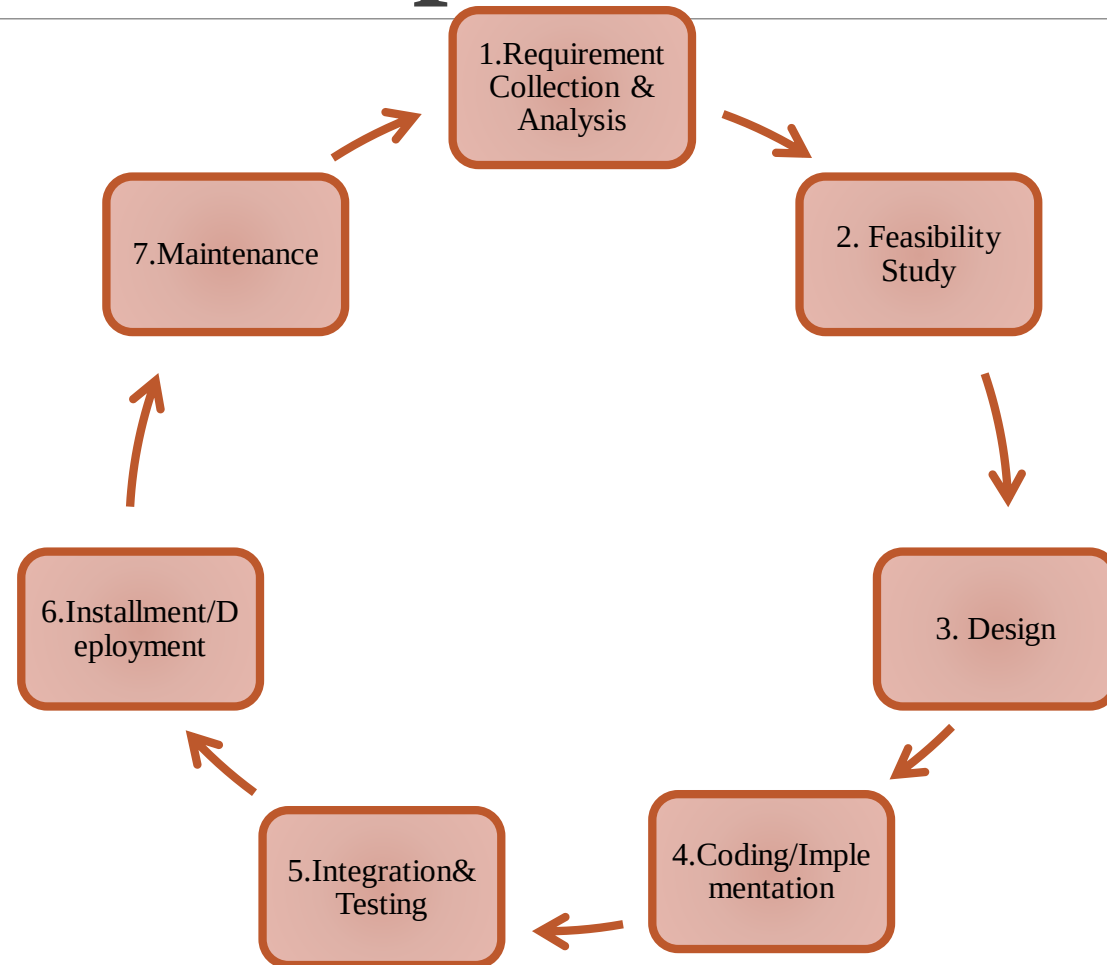


CSE327 (Software Engineering)

Faculty: AKM Iqtidar Newaz (IqN)

LECTURE - 2

Software Development Life Cycle



Software Development Life Cycle (Continued)

Requirement Collection & Analysis or Planning: The requirement is the first phase in the SDLC process. It is conducted by the senior members with inputs from all the stakeholders and domain experts in the industry. Planning for the quality assurance requirements and recognition of the risks involved is also done at this stage. This stage gives a clearer picture of the scope of the entire project and the anticipated issues, opportunities and directives which triggered the project. Requirement gathering stage need teams to get detailed and precise requirements. This helps the companies to finalize the necessary timeline to finish the work of that system.

Software Development Life Cycle (Continued)

Feasibility Study: Once the requirement analysis phase is completed the next SDLC step is to define and document software needs. This process conducted with the help of '**Software Requirement Specification**' document also known as 'SRS' document. It includes everything which should be designed and developed during the project life cycle.

Software Development Life Cycle (Continued)

There are mainly five types of feasibility checks:

- Economic: Can we complete the project within the budget or not?
- Legal: Can we handle this project as cyber law and other regulatory framework/compliances.
- Operation feasibility: Can we create operations which is expected by the client?
- Technical: Need to check whether the current computer system can support the software
- Schedule: Decide that the project can be completed within the given schedule or not.

Software Development Life Cycle (Continued)

Design: In this third phase, the system and software design documents are prepared as per the requirement specification document. This helps define overall system architecture. This design phase serves as input for the next phase of the model.

There are two parts in design phase:

- High Level Design (HDL) - high level design and name of each module
 - an outline about the functionality of every module
 - interface relationship and dependencies between module
 - E-R module.
- Low Level Design (LLD) - functional logic of the modules
 - database tables-type and size
 - complete detail of the interface
 - complete input & output of every module.

Software Development Life Cycle (Continued)

Coding/Implementation: Once the system design phase is over, the next phase is coding. In this phase, developers start build the entire system by writing code using the chosen programming language. In the coding phase, tasks are divided into units or modules and assigned to the various developers. It is the longest phase of the Software Development Life Cycle process. In this phase, Developer needs to follow certain predefined coding guidelines. They also need to use programming tools like compiler, interpreters, debugger to generate and implement the code.

Software Development Life Cycle (Continued)

Integration & Testing: Once the software is complete, and it is deployed in the testing environment. The testing team starts testing the functionality of the entire system. This is done to verify that the entire application works according to the customer requirement. During this phase, QA and testing team may find some bugs/defects which they communicate to developers. The development team fixes the bug and send back to QA for a re-test. This process continues until the software is bug-free, stable, and working according to the business needs of that system.

Software Development Life Cycle (Continued)

Installation/Deployment: Once the software testing phase is over and no bugs or errors left in the system then the final deployment process starts. Based on the feedback given by the project manager, the final software is released and checked for deployment issues if any.

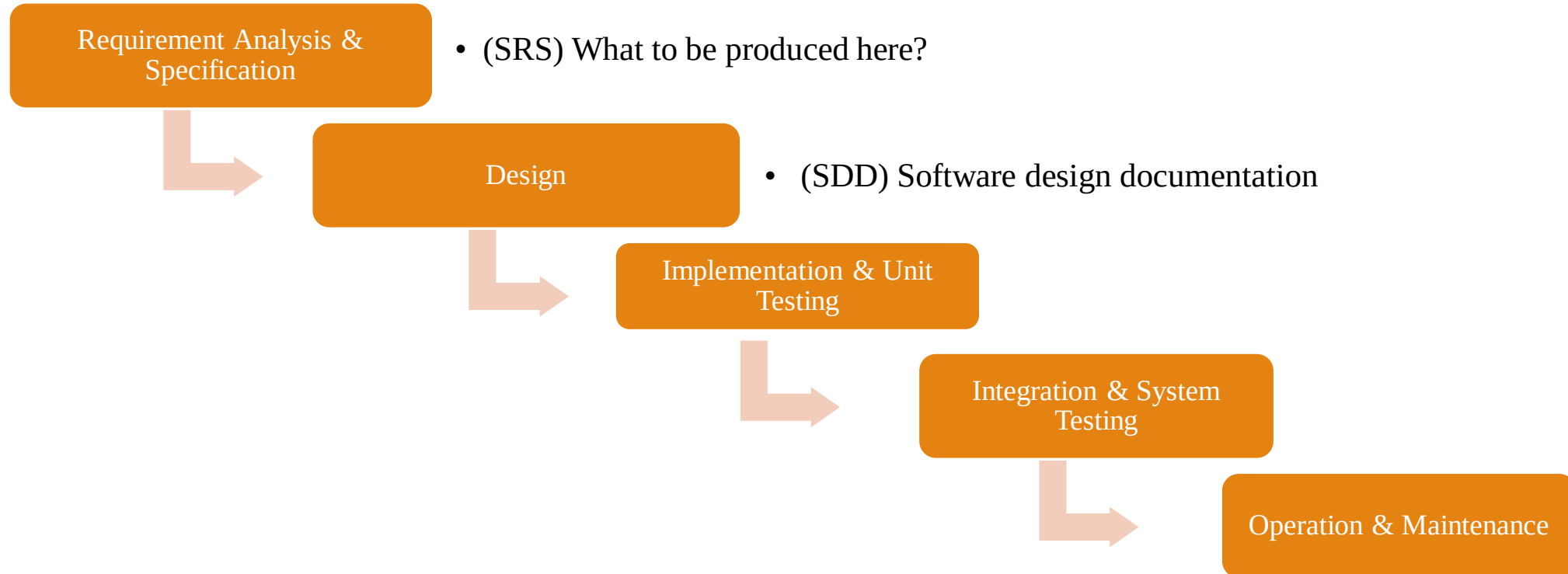
Maintenance: Once the system is deployed, and customers start using the developed system, following 3 activities occur:

Software Development Life Cycle (Continued)

- Bug fixing - bugs are reported because of some scenarios which are not tested at all
- Upgrade - Upgrading the application to the newer versions of the Software
- Enhancement - Adding some new features into the existing software

The main focus of this SDLC phase is to ensure that needs continue to be met and that the system continues to perform as per the specification mentioned in the first phase.

Waterfall Model



Waterfall Model (Continued)

Based on this model all the other updated model came.

Every module has a specific team and only after finishing one module sends it to the next team/module.

When to use Waterfall model?

Requirements are not changing frequently.

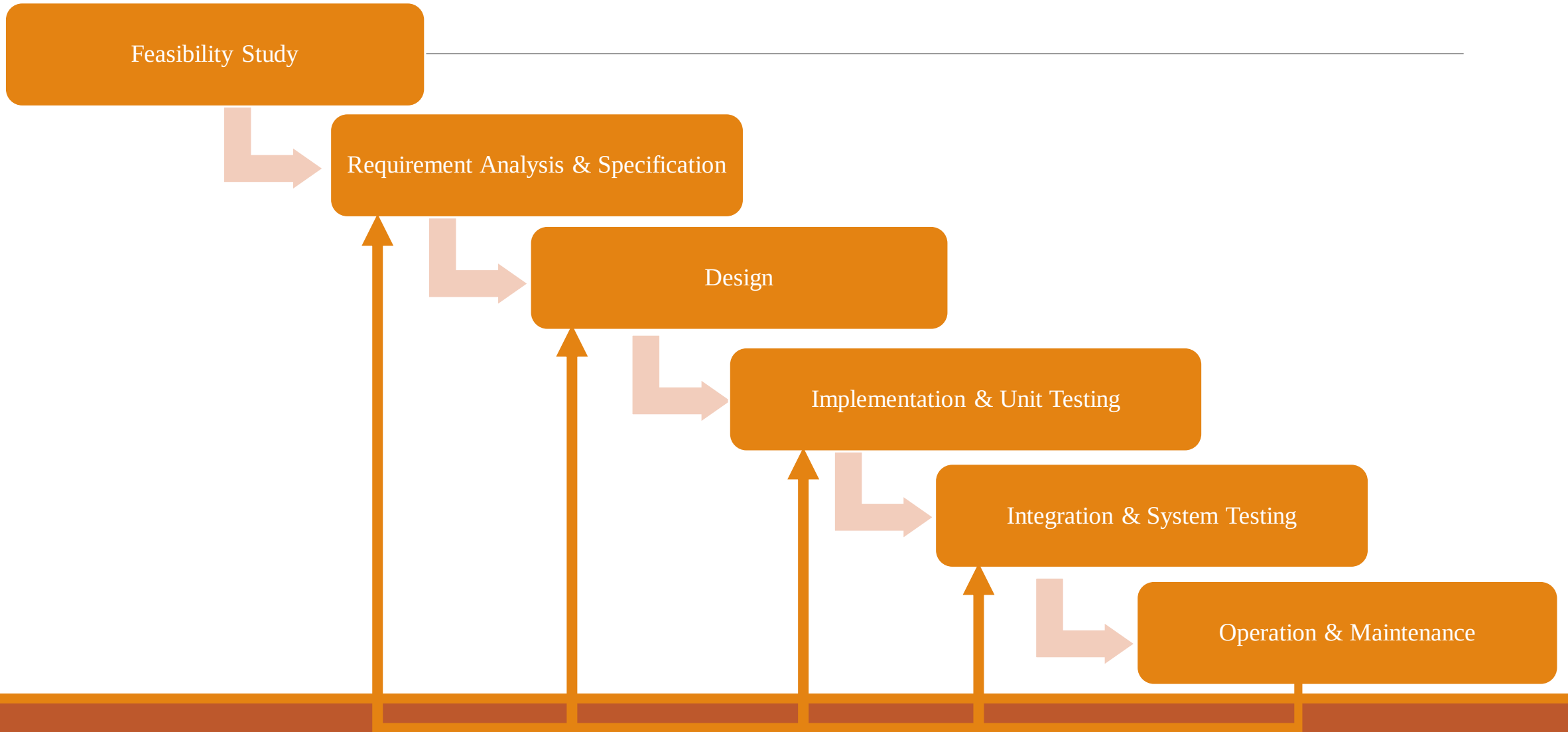
Application is not complicated and big.

Project is short.

Requirement is clear.

Environment is stable.

Iterative Waterfall Model



Iterative Waterfall Model (Continued)

When to use iterative waterfall model

- The requirements are defined and clearly understood.
- New technology is being learned by the development team.
- There are some high-risk features and goals which might in the future.
- Application of Iterative Waterfall Model Major requirements are defined but the minor details might involve when time goes.
- New technologies are being used and there is a learning curve for the programmers to learn.
- Resource are limited to do a huge project as if a small project automates are in contact rather than permanent.
- Very high risk as a goal of the project might change from time to time.

Iterative Waterfall Model (Continued)

Advantage:

- Feedback Path: iterative waterfall allows the mechanism of error correction because there is a feedback path from one phase to its preceding phase which it lacks in the Waterfall Model.
- Simple: iterative waterfall model is simple to understand and use. It is the most widely used software development model evolved so far. Parallel development can be done.

Disadvantage:

- More resource: may be required to implement the iterative waterfall model.
- Difficult to include change requests: In the iterative waterfall model, all the requirements must be clearly defined before starting of the development phase but sometimes customer requirement changes which is difficult to incorporate change requests that are made after development phase starts.
- Not support Intermediate delivery: Project has to be fully completed before it delivered to the customer.
- Risk handling: Project is prone to many types of risk but there is no risk handling mechanism. Not suitable for a small project.